

Accelerating Reasoning Through Step Speculation with Parallel Hidden Representation Verification

Anonymous ACL submission

Abstract

Production deployments of reasoning large language models (LLMs) require step-level verification of an *ever-growing* set of concepts: new safety policies, factuality requirements, formatting constraints, and task-specific correctness criteria emerge continuously over a model’s lifetime. Existing verifiers handle this regime poorly: trained process reward models (PRMs) require $\mathcal{O}(10^5)$ samples per retrain and catastrophically forget prior concepts, while LLM-as-judge baselines incur inference cost that grows linearly with the number of monitored concepts. We propose *Latent Activation Verification Assistance* (LAVA), a continual-learning framework for step-level verification of LLM reasoning. LAVA trains lightweight probes on the frozen backbone’s hidden activations; each new concept becomes a new probe trained on $\sim 1,000$ samples in minutes, while existing probes remain frozen — giving *zero backward interference by construction*, constant per-task training cost, and bounded inference-latency growth as the number of probes grows. We evaluate LAVA along the standard continual-learning axes (sequential concept arrival, within-concept distribution shift, base-model drift, and concept-count scaling) and compare against regularization-based (EWC, LwF), replay-based, and parameter-isolation continual-learning baselines, as well as against LLM-as-judge and sequentially retrained PRMs. As a downstream application, the resulting probe bank serves as a parallel multi-concept verifier in speculative reasoning pipelines, delivering 69% reduction in token usage and $3\text{--}5\times$ speedup in inference latency at matched task quality.

1 Introduction

Production deployments of reasoning LLMs are intrinsically *non-stationary*: the set of concepts a verifier must monitor grows over time as new safety policies are issued, new task domains are added,

attackers discover new jailbreak families, and the base model itself is updated through continued pre-training or RLHF. A useful verifier therefore needs more than high accuracy on a fixed benchmark suite — it needs to *accumulate* new concepts cheaply, without forgetting the old ones and without paying a recurring inference tax for every concept added.

The existing tooling for step-level verification was not designed for this regime. Process reward models (PRMs) (Liu et al., 2025) achieve strong per-concept accuracy but require $\mathcal{O}(10^5)$ annotated samples to converge, and when fine-tuned sequentially on new concepts they suffer catastrophic forgetting — a problem extensively documented in the continual learning literature (Kirkpatrick et al., 2017; Li and Hoiem, 2017; Lopez-Paz and Ranzato, 2017). LLM-as-judge approaches such as SpecReason (Pan et al., 2025) sidestep training entirely but pay an inference cost that scales linearly with the number K of concepts being checked; with K in the tens, the cumulative prompting becomes the dominant latency term. Multi-head joint training of a single shared verifier requires simultaneous access to all per-concept data, which is incompatible with a streaming arrival of concepts.

We propose *Latent Activation Verification Assistance* (LAVA), a continual-learning framework for step-level verification. LAVA keeps the LLM backbone and feature extractor frozen, and represents the verifier as a *bank of independent per-concept probes* trained on $\sim 1\text{k}$ samples each. When a new concept arrives, we simply add a new probe; prior probes are never touched. This architectural choice yields three properties *by construction*, without any auxiliary regularizer or replay buffer (see Section 6.3 for the formal statements):

1. **Zero backward interference** (Prop. 6.1): the accuracy on every previously learned concept is unchanged after any number of new concepts have been added, so BWT = 0.
2. **Constant per-task training cost** (Prop. 6.2):

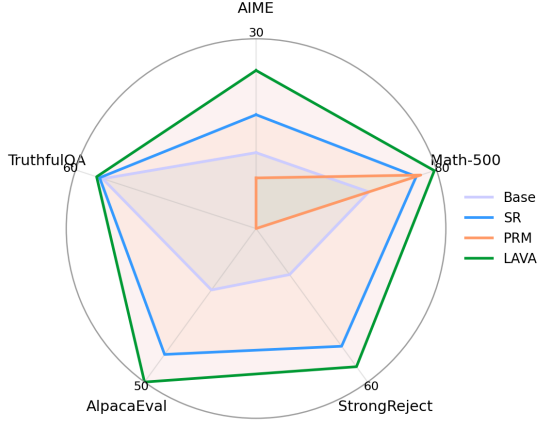


Figure 1: Performance comparison given a fixed token budget. SR is short for SpecReason (Pan et al., 2025). Base model is Qwen2.5-1.5B-Instruct, and PRM is a process reward model QwenMathPRM.

adding the k -th concept always costs $\mathcal{O}(1)$ samples and minutes of compute, independent of k .

3. **Bounded inference overhead** (Prop. 6.3): activating K probes adds only $\mathcal{O}(Kd)$ work per step, orders of magnitude below one transformer forward pass.

The empirical behavior on a fixed token budget is previewed in Figure 1.

Contributions.

1. We reformulate step-level verification of LLM reasoning as a continual-learning problem (Section 6.1–6.2), and articulate the precise axes on which PRMs and LLM-as-judge break down in this regime.
2. We show that the LAVA probe bank is a natural continual-learning system: a frozen backbone plus independent per-concept probes provides zero forgetting, constant per-task cost, and flat inference scaling *by construction* (Propositions 6.1–6.3).
3. We provide a continual-learning evaluation protocol (Section 6.5) covering sequential concept arrival, within-concept distribution shift, base-model drift, and concept-count scaling, with comparisons against regularization-based (EWC, LwF), replay-based, and parameter-isolation baselines in addition to LLM-as-judge and retrained PRMs.
4. As a downstream application, we deploy the continually trained probe bank as the parallel multi-concept verifier of a speculative reasoning pipeline, recovering the efficiency gains (69%

token reduction, 3–5 $\times$ latency speedup) over LLM/PRM-based verification reported in prior work, now obtained as a consequence of the CL formulation rather than a one-off design.

2 Related Work

Probes in Large Language Models. Recent work has demonstrated the particular utility of probes (Alain and Bengio, 2016) in analyzing large language models (LLMs) interpretability. Notably, probes are used as a signal for LLM safety, defense or telling a lie (Mallen et al., 2023; Zou et al., 2023; Yi et al., 2025); probes as a sign for explaining reasoning and logic truthfulness, and LLM sycophancy (Manigrasso et al., 2024; Zhang et al., 2025; Papadatos and Freedman, 2024). Probe accuracy has deep relation with LLM uncertainty (Wang et al., 2025; Dakhmouche et al., 2025), LLM answering accuracy (PROBES, 2025), and copyright data (Ma et al., 2024). These researches has proven that probes is a powerful method for eliciting latent knowledge, and showing a signal for a given concept (Ju et al., 2024). Motivated by recent findings on probe-based analysis, we develop a pipeline for training lightweight probes as verifiers and deploying them in parallel.

Reasoning with Ensemble LLMs. There is a trend (Miao et al., 2025) to use multiple LLMs, where easy questions using a small/fast model, and harder question using a large/slow model. The most common paradigm is to let model generate a candidate span (which can be seen as a step in the reasoning path), and then let another LLM to evaluate all the candidates to finally get the best candidate (Lv et al., 2024; Xu et al., 2025; Liu et al., 2024). Most works use majority vote, or verification using consistency (Chen et al., 2023). Smoothie (Guha et al., 2024) uses generation embedding to compute score for each generation and select the best model. System 1.x (Saha et al., 2025) uses a trained classifier to determine whether the question is easy or hard and then send to different planner.

However, existing approaches incur substantial computational overhead in the verification step, relying on LLM-based evaluation or consistency-based methods, and — crucially for the continual setting of this work — they are not designed to absorb a stream of new concepts without retraining or per-concept prompting overhead. In contrast, we employ lightweight probes as efficient, modular, continually extensible verifiers; we defer a detailed

comparison to the continual-learning literature to Section 6.6.

3 Latent Activation Verification Assistance (LAVA)

3.1 Problem Setup

We define the reasoning task over a finite vocabulary $\mathcal{V}$. A reasoning trajectory r is a sequence of n discrete steps $r = (r_1, r_2, \dots, r_n)$, where each step $r_i \in \mathcal{V}^*$ (Kleene star over vocabulary) is a variable-length sequence of tokens terminated by a predefined delimiter (e.g., a newline or a `<|thought_end|>` tag). Let $x \in \mathcal{V}^*$ denote the input query. We assume access to (i) a *draft* autoregressive model M_w (lower latency) and (ii) a *strong* model M_s (higher capability), with the goal of achieving strong-model-level quality at reduced compute by selectively invoking M_s . We summarize the notation we use in Appendix A.

Assumptions and constraints. We assume an *offline* setting where probes (the verifier) can be trained using supervision derived from labels from human preferences, or oracle reward model signals (details in §3.2.4). At inference, we aim to minimize calls to M_s while keeping acceptance errors (accepting a bad step) below a target rate.

High-Level Overview and Motivation. Cascaded and speculative reasoning pipelines reduce compute by letting a weak model propose intermediate steps, but conventional verification often requires an additional expensive scoring pass (or generation) by the strong model. Many step-level properties needed for verification (e.g., correctness/safety/compliance) may be partially predictable from the model’s latent activations for the proposed step, enabling a low-cost verifier that avoids extra strong-model computation.

3.2 Core method: Latent Activation Verification

We introduce **LAVA** (Latent Activation Verification Assistance): a bank of lightweight probes operating on a fixed hidden-state representation of each proposed step. The probes produce per-concept scores that gate whether the pipeline accepts the draft step or triggers strong-model regeneration.

Pipeline (training vs. inference). At inference, M_w proposes a step r_{i+1}^w , LAVA extracts a feature vector f from hidden states for that step, and the bank of K concept-specific probes $\{P_k\}_{k=1}^K$

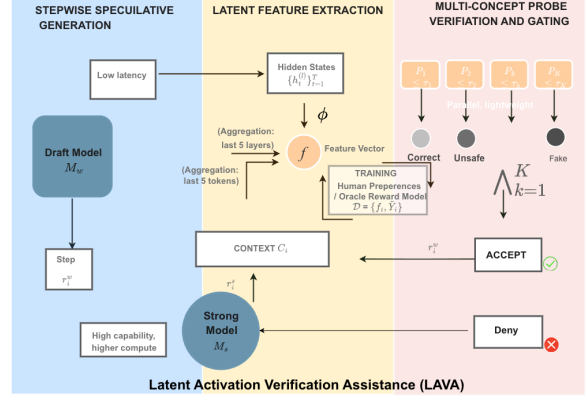


Figure 2: **Latent Activation Verification.** A draft model proposes each reasoning step; LAVA verifies the step using parallel latent probes and threshold gating; on failure, the strong model regenerates the current step.

(one probe per verification concept, e.g. correctness, safety; formally defined in Eq. 4) emits scores $(P_k(f))_{k=1}^K$ in a single forward pass. The step is then either accepted or replaced by a step from M_s . During training, the probes are fit to predict concept alignment/teacher judgments from f , and thresholds $\{\tau_k\}$ are calibrated to control acceptance/false-accept rates.

An overview of the resulting inference pipeline is shown in Figure 2. We present LAVA in three components: (1) stepwise speculative generation, (2) latent feature extraction, and (3) multi-concept probe verification and gating.

3.2.1 Component 1: Stepwise Speculative Generation

Given context $C_i = [x, r_1, \dots, r_i]$, the draft model proposes the next step

$$r_{i+1}^w \sim M_w(\cdot \mid C_i). \quad (1)$$

If verification fails (defined in §3.2.3), we invoke the strong model to regenerate the step:

$$r_{i+1}^s \sim M_s(\cdot \mid C_i). \quad (2)$$

Why stepwise? Gating at step boundaries limits error propagation: an invalid step can be corrected before it contaminates later steps. The delimiter we used is `\n\n`.

3.2.2 Component 2: Latent Activation Extraction

Let M be the backbone whose activations are used for verification (typically M_w , but other choices, including M_s or an assistance model are possible).

For a candidate step r_{i+1}^w of length T tokens, denote the hidden state at layer $l \in \{1, \dots, L\}$ and token position $t \in \{1, \dots, T\}$ as $h_t^{(l)} \in \mathbb{R}^d$. We define a step-level feature vector

$$f := \phi\left(\{h_t^{(l)}\}_{t=1}^T\right) \in \mathbb{R}^d, \quad (3)$$

where $\phi(\cdot)$ is an aggregation operator, e.g. vector concatenation, pooling, etc. In the simplest instantiation (used in our draft), we choose the last layer l^* and take the concatenation of the last 5 token representations:

$$h_{T-4}^{(l)} \oplus h_{T-3}^{(l)} \oplus \dots \oplus h_T^{(l)},$$

where a single step often contains a localized claim or transformation; the terminal token representation can encode summary information about whether the step “fits” the context and internal constraints.

3.2.3 Component 3: Multi-Concept Probe Verification and Gating

We verify each candidate step against K concepts (e.g., correctness, safety, format compliance). For each concept k , we train a probe

$$P_k : \mathbb{R}^d \rightarrow [0, 1], \quad P_k(f) = \sigma(w_k^\top f + b_k), \quad (4)$$

where σ is the logistic sigmoid and (w_k, b_k) are learnable parameters.¹

The verifier outputs a score vector $y \in [0, 1]^K$:

$$y = [P_1(f), \dots, P_K(f)]^\top. \quad (5)$$

We accept the draft step if all concept constraints pass:

$$\text{ACCEPT}(r_{i+1}^w) \iff \bigwedge_{k=1}^K (P_k(f) \geq \tau_k). \quad (6)$$

Otherwise we reject and regenerate using M_s (Eq. 2). The final step is

$$r_{i+1} = \begin{cases} r_{i+1}^w & \text{if } \text{ACCEPT}(r_{i+1}^w) \\ r_{i+1}^s & \text{otherwise.} \end{cases} \quad (7)$$

We illustrate this inference algorithm in Appendix B.

Complexity. Probe inference is $O(Kd)$ per step and typically negligible relative to a Transformer forward pass. The expected compute reduction depends on the acceptance rate and the relative cost ratio between M_s and M_w .

¹We use multi-layer probes with regularization.

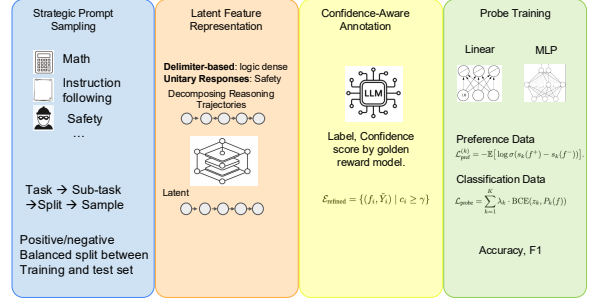


Figure 3: Automated Probe Construction Pipeline.

3.2.4 Training Objective and Optimization

This section specifies how probes (Bengio and LeCun, 2007) $\{P_k\}$ and thresholds $\{\tau_k\}$ are learned.

Probe Supervision and Datasets We assume a training set of step-level examples $\{(x^{(j)}, C_i^{(j)}, r_{i+1}^{w,(j)}, z^{(j)})\}$ where $z^{(j)} \in \{0, 1\}^K$ encodes concept labels for the candidate step. Labels may come from: (i) human annotations, (ii) automated rules/tests (e.g., correct or not by pattern matching), or (iii) teacher judgments from a stronger verifier (potentially M_s or an golden oracle).

Loss Functions Given features $f^{(j)}$ extracted from candidate step $r_{i+1}^{w,(j)}$, we train probes with a multi-label logistic loss:

$$\mathcal{L}_{\text{probe}} = \sum_{k=1}^K \lambda_k \cdot \left(-z_k \log P_k(f) - (1 - z_k) \log(1 - P_k(f)) \right), \quad (8)$$

where $\lambda_k \geq 0$ weights concepts (e.g., to upweight safety).

Preference-based supervision. If only pairwise preferences are available (e.g., $r^+ \succ r^-$), we fit a probe score $s_k(f)$ and use a Bradley–Terry objective:

$$\mathcal{L}_{\text{pref}}^{(k)} = -\mathbb{E}[\log \sigma(s_k(f^+) - s_k(f^-))]. \quad (9)$$

For paired data, we use this preference-based supervision; for single-entry data, we use the logistic loss.

4 Automated Probe Construction Pipeline

We propose an automated pipeline to operationalize concept-specific probing across diverse reasoning domains. The procedure is illustrated in Figure 3 which systematically maps raw task instances to a curated latent-feature space $\mathcal{X} \times \{0, 1\}$.

4.1 Data Construction and Labeling

We formalize a task as a tuple $(\mathcal{D}^{(k)}, \mathcal{C}^{(k)})$, where $\mathcal{D}^{(k)}$ is a domain-specific dataset and $\mathcal{C}^{(k)}$ is the target semantic concept (e.g., *mathematical correctness*, *safety status*, or *stylistic quality*).

Strategic Prompt Sampling. For each task k , we define a prompt space $\mathcal{D}_{\text{prompts}}^{(k)} \subset \mathcal{V}^*$. To ensure the probe captures nuanced decision boundaries, we employ a **stratified sampling scheme** without replacement. Strata are defined by meta-labels (e.g., difficulty, safety status, or writing quality) to ensure the resulting dataset $\mathcal{X}^{(k)}$ is balanced across the concept’s failure and success modes.

Decomposing Reasoning Trajectories. Let $x \in \mathcal{D}_{\text{prompts}}^{(k)}$ be an input prompt. A reasoning trajectory is modeled as a sequence of atomic units (steps) $r = (r_1, \dots, r_N)$, where each $r_i \in \mathcal{V}^*$. For datasets lacking native step-level annotations, we employ a **synthetic trace generation** strategy using Qwen2.5-Instruct-7B. We induce structure via system prompting that enforces Chain-of-Thought (CoT) reasoning.

- **Granularity:** In logic-dense domains (Math, QA), we apply delimiter-based parsing.
- **Unitary Responses:** For safety-critical concepts where responses are typically succinct, the entire generation is treated as a single semantic step ($N = 1$).

Latent Feature Representation. For each step r_i , we extract a high-dimensional representation $f_i \in \mathbb{R}^d$ by aggregating its internal hidden states $\{h_t^{(l)}\}_{t=1}^T$ from the backbone model with the pooling operator $\phi(\cdot)$ defined in Eq. 3 (Section 3.2.2).

Denoising via Confidence-Aware Annotation. Each step is assigned a binary label $\tilde{Y}_i \in \{0, 1\}$ via an **oracle-distillation** process. We model the observed label as a realization of a noisy channel: $\tilde{Y}_i \sim \text{Annotator}_k(Y_i; \theta_k)$, where Y_i is the latent ground truth. To mitigate the influence of “hallucinated” or ambiguous labels from our automated judges (e.g., Claude-3.7-Sonnet for Math, Llama-3-Guard for Safety), we utilize a **confidence-thresholding filter**. Given a scalar confidence score $c_i \in [0, 1]$ provided by the annotator, we retain only the subset:

$$\mathcal{E}_{\text{refined}} = \{(f_i, \tilde{Y}_i) \mid c_i \geq \gamma\} \quad (10)$$

Setting $\gamma = 0.8$ effectively prunes the “aleatoric noise” inherent in automated labeling, yielding a high-fidelity training signal for the probes. The refined set $\mathcal{E}_{\text{refined}}$ is then used to train each probe with the multi-label logistic objective in Eq. 8.

4.2 Evaluation Suite and Benchmarks

We evaluate our pipeline across four distinct axes of LLM behavior, utilizing state-of-the-art benchmarks (summarized in Table 1):

Reasoning: MATH (Lightman et al., 2023) consists of competition-level mathematics problems with step-by-step solutions, designed to evaluate multi-step symbolic reasoning. AIME (Zhang et al., 2024) contains problems from the American Invitational Mathematics Examination, emphasizing concise but nontrivial mathematical reasoning under strict answer formats. GSM8K (Cobbe et al., 2021) is a collection of grade-school math word problems requiring multi-step arithmetic and logical reasoning.

Safety: WildJailbreak (Shen et al., 2024) is a large-scale benchmark of adversarial prompts collected from real-world jailbreak attempts, designed to test a model’s robustness against safety policy violations. StrongReject (Souly et al., 2024) focuses on high-confidence unsafe requests and evaluates whether models can consistently refuse harmful instructions while maintaining appropriate refusal behavior.

Truthfulness: TruthfulQA (Lin et al., 2021) measures a model’s tendency to produce truthful answers in the presence of common misconceptions, particularly in domains where incorrect but plausible responses are prevalent.

Instruction Following: AlpacaEval (Dubois et al., 2024) evaluates instruction-following quality via pairwise model comparisons judged by a reference LLM, covering a diverse set of open-ended instructions and response styles.

To ensure robust generalization, we split datasets into fixed-size train/test sets using stratified sampling. To maximize the diversity of the probe’s exposure, we employ sampling *with replacement* when generating multiple model responses for a single prompt, ensuring the probe learns to distinguish concepts across a wide variance of linguistic realizations.

4.3 Probe Architecture and Training

To identify the optimal trade-off between expressivity and generalization, we investigate two archi-

Table 1: Evaluation Suite and Benchmarks.

Task	Sub-task	Split	Labeler
Math Reasoning	MATH	[1000, 100]	LLM
	AIME	[833, 100]	LLM
	GSM8K	[1000, 100]	Human
Truthfulness	TruthfulQA	[707, 100]	Human
Safety	WildJailbreak	[500, 100]	LLM
	StrongReject	[250, 50]	LLM
Instruction Following	AlpacaEval	[701, 100]	LLM

tectural paradigms for our probes, following the methodology of Zhang et al. (2025):

Linear Probe. A minimalist configuration consisting of a single linear transformation $W \in \mathbb{R}^{d \times 1}$, mapping the latent feature dimension (e.g., $d = 4096$ for Llama-3.1-8B) directly to an output logit. To ensure stable convergence, we employ Xavier uniform initialization with a conservative gain of 0.1 and zero-initialize the bias terms.

MLP Probe. A deeper architecture designed to capture non-linear interactions within the latent space. The MLP consists of two hidden layers with dimensions [512, 256], utilizing ReLU activations and dropout regularization ($p = 0.2$) to mitigate overfitting.

Implementation Details. The probes are optimized using the Adam optimizer with a learning rate of 5×10^{-3} , momentum of 0.9, and weight decay of 1×10^{-4} . For probe loss, we utilize BCEWithLogitsLoss for numerical stability. Training is conducted with a batch size of 64 for 50 to 200 epochs, incorporating gradient clipping (max norm = 1.0) to ensure trajectory stability. In inference, a classification threshold of 0.5 is applied to map probabilities to the binary concept space.

4.4 Probe Performance Analysis

The efficacy of our automated construction pipeline is reflected in the high classification accuracy across diverse concept domains (Table 2). **Acc(%)** describes the accuracy of the predicted label on the test split, **F1 Score** calculates the harmonic mean of precision and recall on the negative data (incorrect, unsafe). For preference data, the probe is trained using Equation (9), otherwise the loss is calculated as in Equation (8).

Our empirical results demonstrate that the MLP probes successfully distill complex semantic concepts from internal representations. Notably, we

Table 2: Probe performance.

Sub-task	Loss Type	Probe	Acc (%)	F1
MATH	Correctness	MLP	89	82
		Linear	72	67
AIME	Correctness	MLP	88	84
		Linear	72	68
GSM8K	Preference	MLP	92	84
		Linear	75	68
TruthfulQA	Truthfulness	MLP	82	70
		Linear	74	68
WildJailbreak	Safety	MLP	80	71
		Linear	68	52
StrongReject	Safety	MLP	78	69
		Linear	64	50
AlpacaEval	Preference	MLP	90	74
		Linear	80	69

achieve superior performance in structured reasoning tasks such as *GSM8K* (92%) and *Alpaca_Eval* (90%), suggesting that preference and correctness signals are strongly encoded in the model’s latent trajectories. While safety concepts (*WildJailbreak*, *StrongReject*) present a more challenging decision boundary, the probes still maintain robust predictive power ($\geq 78\%$), validating the effectiveness of the confidence-aware denoising and stratified sampling strategy. Across all sub-tasks, the MLP probe consistently outperforms the linear probe in both accuracy and F1, indicating that the target signals are more cleanly separated with a non-linear readout.

5 Experiments

5.1 Single-Concept Verification in Speculative Reasoning

Experimental Configuration. We evaluate the efficacy of our probe-based verification within a speculative reasoning framework. In this setting, a draft model (M_w) generates reasoning steps, while a verification mechanism determines whether to accept the step or trigger a correction from a stronger model (M_s).

Implementation Details. We implement our system on top of vLLM. To enable parallel hidden-representation verification, we make a lightweight modification to the model forward pass so that the required hidden states can be returned during generation (for downstream probe evaluation and gating). While exporting hidden states introduces additional overhead, vLLM remains substantially faster than a the transformers implementation in our setup even under this mode. Therefore, we adopt the vLLM

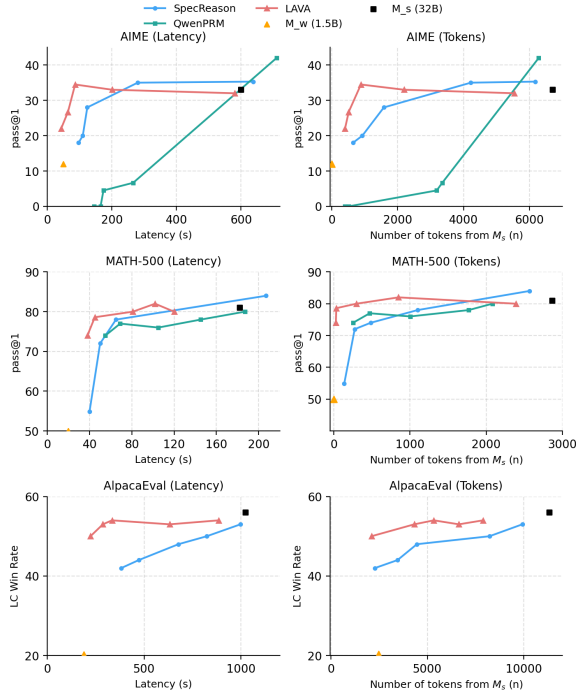


Figure 4: Performance and cost trade-off on functional benchmarks, including AIME, MATH-500, and AlpacaEval.

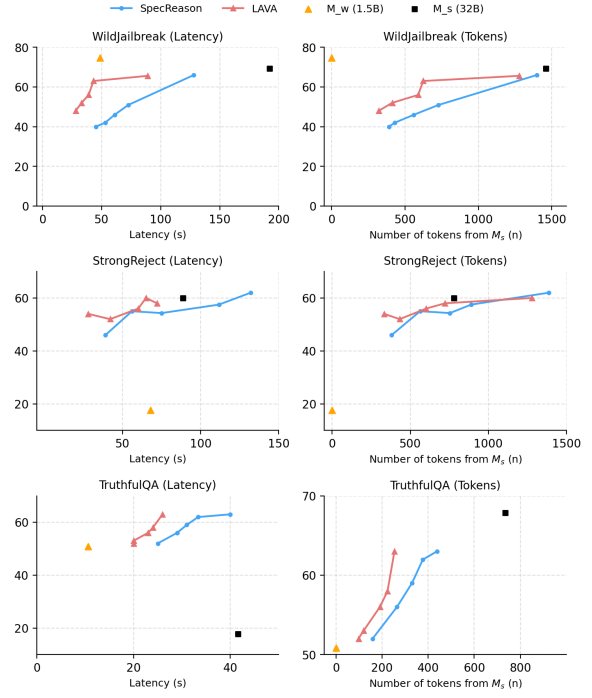


Figure 5: Performance and cost trade-off on safety/truthfulness benchmarks, including WildJailbreak, StrongReject, and TruthfulQA.

implementation as our default system backend.

Models and Baselines. We utilize a diverse suite of models: **Small-scale** (M_w) including Deepseek-Distill-Qwen-1.5B, Qwen2.5-1.5B-Instruct; and **Large-scale** (M_s) including QwQ-32B, Qwen-2.5-32B. Our method is compared against two competitive baselines: (1) **SpecReason** (Pan et al., 2025), which prompts M_s directly for step-wise verification, and (2) **Off-the-shelf PRMs**, specifically Qwen-Math-PRM-7B. Each data point corresponds to a distinct threshold value $\tau \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. For SPECReason, the threshold is specified via an integer confidence level $c \in \{1, 3, 5, 7, 9\}$ provided in the prompt, where each value corresponds to the respective τ (i.e., $c = 10\tau$). The PRM threshold β is applied to the normalized output score from the final classification head of the PRM, $\beta \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$.

Results and Comparative Analysis. Our results, illustrated in Figure 4 and Figure 5, we could draw the following conclusion. 1) **Computational Efficiency and Latency:** On MATH-500 and AIME, our method achieves a **69% reduction in tokens** sampled from the large model compared to PRM-based verification. Furthermore, we observe a **3 $\times$ to 5 $\times$ speedup** in end-to-end latency

relative to SpecReason. 2) **Domain-Specific Robustness:** In mathematical reasoning, our method significantly outperforms both the base model and SpecReason. In safety benchmarks, while the probe-based approach maintains the base model’s inherent safety guardrails, it demonstrates superior performance over the SpecReason baseline. 3) **Generalization Boundaries:** Evaluation on WildJailbreak indicates a performance plateau. A plausible explanation is that safety does not scale monotonically with model size: increasing parameter count does not necessarily yield safer behavior, and in our results the 32B model exhibits safer performance on WildJailbreak than some smaller counterparts.

Ablation on latent activation from M . We could extract latent activation from the M_w , M_s or any off-the-shelf model. Table 3 shows that embeddings extracted from larger models substantially outperform smaller models in this setting. However, using larger models for latent activation extraction yields higher latency. Using M_w (Qwen2.5-1.5B) already achieves strong results (Acc= 89, F1= 82, MATH score= 80), while scaling to larger models yields a consistent improvement, most notably on the downstream MATH score (Acc= 91, F1= 82, MATH score= 82).

Overall, these results highlight the importance of model-family alignment for representation quality and indicate a modest but reliable benefit from using a larger backbone for feature extraction.

Table 3: Comparison of different embedding models.

	Acc (%)	F1	MATH score
Qwen2.5-1.5B	89	82	80
Qwen2.5-32B	91	82	82
LLaMA-3-1B	81	71	72
Gemma-3-27B	91	84	79

Internal Representation Dynamics. Table 4 indicates that the aggregation strategy over the last four layers has a substantial effect on probe quality. Using a single-layer representation (L) provides a moderate baseline (Acc= 79, F1= 72, MATH score= 76). Among the $L - 4:L$ variants, $\phi = \text{pooling}$ underperforms this baseline (Acc= 74, F1= 68, MATH score= 72), suggesting that pooling may wash out layer-specific signals that are useful for discrimination. In contrast, $\phi = \text{concat}$ achieves the strongest results by a large margin (Acc= 89, F1= 82, MATH score= 80), implying that preserving per-layer information yields richer features for the classifier. Finally, $\phi = \text{min}$ performs worst (Acc= 70, F1= 56, MATH score= 62), consistent with an overly conservative operator that is dominated by the least informative layer and discards complementary evidence. Overall, these results suggest that multi-layer information is beneficial, but only when the aggregation retains diversity (e.g., concatenation) rather than aggressively compressing it.

Table 4: Comparison of different aggregation methods.

l	Acc	F1	MATH score
L	79	72	76
$L - 4 : L$ ($\phi = \text{pooling}$)	74	68	72
$L - 4 : L$ ($\phi = \text{concat}$)	89	82	80
$L - 4 : L$ ($\phi = \text{min}$)	70	56	62

Transferability. We train the probe using AIME training set and GSM8k training set, test on MATH-500 probe test set and MATH benchmark. As shown in Table 5, the probe trained on AIME exhibits strong transferability, whereas the probe trained on GSM8k transfers poorly. We attribute

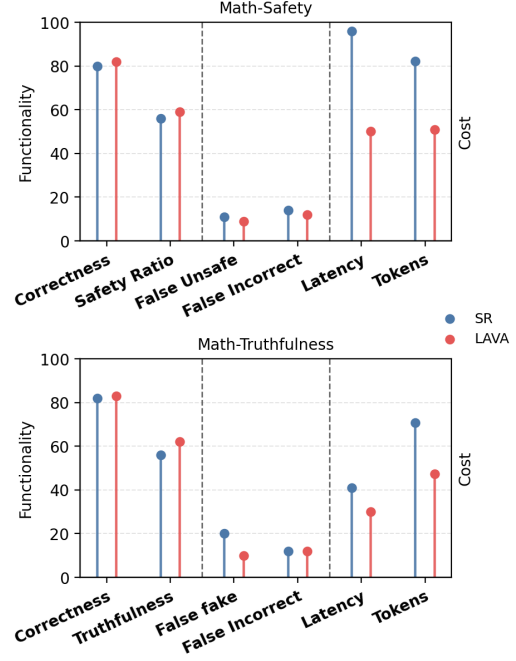


Figure 6: Parallel multi-concept verification. Higher values are preferable in the left panel, whereas lower false negative rates (middle) and reduced latency and token usage (right) indicate better practical utility and lower computational cost, respectively. LAVA consistently outperforms all baselines across these metrics.

this gap primarily to differences in data distribution and reasoning structure: GSM8k typically involves short, relatively homogeneous reasoning traces, while MATH and AIME feature longer, more compositional and inductive solutions with formats that are more mutually consistent.

Table 5: Transferability of probes.

Training Set	Acc	F1	MATH score
AIME	84	76	78
GSM8k	72	64	58
MATH	89	82	80

5.2 Parallel Multi-Concept Verification

Setup. Real-world deployment requires simultaneous monitoring of multiple attributes (e.g., correctness and safety). We simulate this via a 50/50 mixture of Math-500 and StrongReject, 50/50 mixture of Math-500 and TruthfulQA. We compare against a multi-expert SpecReason baseline that prompts M_s as a specialized verifier for each concept.

Decision Boundary Refinement and Domain Adaptation. Preliminary evaluations, as pre-

sented in Table 10, indicate that a naive deployment of probes trained via the procedure in Section 5.1 yields suboptimal performance. We hypothesize that this degradation stems from an elevated *False Negative* rate, where the probe fails to generalize the concept of “safety” or “correctness” when encountered in out-of-distribution contexts. To mitigate this, we propose a **Latent Landscape Reshaping** strategy. This approach regularizes the feature space by augmenting the training set with objective-aligned samples from disparate domains. Specifically, during the optimization of the safety-concept probe, we incorporate mathematical reasoning trajectories as categorical “positive” (safe) instances. This cross-domain calibration enforces a more robust decision boundary, ensuring the probe distinguishes between concept-specific violations and general domain shifts, ultimately resulting in the enhanced metrics reported in our final results.

Performance. As detailed in Figure 6, our method demonstrates clear advantages in multi-objective scenarios: 1) **Superior Throughput:** Our approach reduces latency by **47.9%** (100s vs. 192s) and total token consumption by **38.1%** (762 vs. 1232 tokens) compared to the SpecReason baseline. 2) **Precision-Recall Balance:** Our method improves the Safety Ratio to **59%** while maintaining identical Correctness (82% pass@1). Although our method exhibits a higher False Unsafe rate (19% vs. 11%), it demonstrates a lower False Incorrectness rate (12% vs. 14%), indicating a more permissive but safer reasoning trajectory. 3) **Architectural Scalability:** Unlike SpecReason, which scales linearly in cost with the number of concepts (due to repeated LLM calls), our probe-based verification allows for parallel attribute checking within a single forward pass.

5.3 Discussions

This work serves as a preliminary investigation into the potential of internal model activation for reasoning verification. We seek to provide answers to two fundamental questions that can guide the future of efficient inference: 1) **Explicit Prompting vs. Implicit Probing:** Is the high latency of autoregressive, “explicit” verification a necessary cost for accuracy? While current benchmarks rely on generative judges, we demonstrate that *implicit* signals within the latent manifold provide a sufficiently high-fidelity signal for verification, suggesting that models “know” the validity of their reasoning long before a transition to the logit space occurs. 2)

Monolithic Verifiers vs. Modular Ensembles: Is a single, high-parameter “golden” reward model superior to a collective of lightweight modules? Our findings challenge the necessity of monolithic PRMs, showing that an ensemble of specialized, 1,000-sample probes can match the performance of models with orders of magnitude more parameters. As LLMs scale in size and cost, this modular probing approach offers a sustainable and agile alternative for multi-dimensional evaluation (e.g., safety, logic, and style) in real-time environments.

6 LAVA as a Continual Verification System

Section 3 introduced LAVA as a probe bank used inside a single speculative-reasoning pipeline. We now reframe the same construction as a *continual learning* (CL) system for step-level verification: the set of concepts being monitored is not fixed in advance, but grows over the lifetime of the deployed model. This section formalizes that setting, states the structural guarantees that the LAVA architecture provides “for free,” and describes our continual evaluation protocol.

6.1 Why Continual Learning?

Production deployments of reasoning LLMs face an inherently non-stationary verification landscape:

- **New safety policies and compliance rules** are issued continuously (new jailbreak families, new regulated topics, new jurisdictional constraints).
- **New task domains** arrive as the deployment scope expands (e.g. adding code review on top of math tutoring).
- **Distributional drift within a concept** occurs as users, attackers, and the world change.
- **The base LLM itself is updated** (continued pre-training, RLHF rounds, instruction-tuning), which can invalidate any verifier trained against the previous checkpoint.
Existing verifiers handle this regime poorly:
- **Trained PRMs** require $\mathcal{O}(10^5)$ annotated samples per retrain and, when fine-tuned sequentially on new concepts, suffer catastrophic forgetting (Kirkpatrick et al., 2017; Li and Hoiem, 2017).
- **LLM-as-judge** avoids forgetting by re-prompting, but inference cost grows linearly in the number of monitored concepts K .
- **Multi-head joint training** requires simultaneous access to data from all concepts, which is

incompatible with a streaming concept arrival.

6.2 Problem Formulation

We consider a stream of K verification tasks (concepts) $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_K$ arriving sequentially. Each task $\mathcal{T}_k$ is associated with a small labeled dataset $\mathcal{D}_k = \{(f_n^{(k)}, z_n^{(k)})\}_{n=1}^{N_k}$ of latent-feature/label pairs (typically $N_k \approx 1,000$, generated by the pipeline of Section 4.2). At time k , a learner may use $\mathcal{D}_k$ and any previously stored parameters, but cannot revisit $\mathcal{D}_{<k}$.

A continual verifier is the resulting tuple $V_K = (\theta_1, \dots, \theta_K)$ deployed at the end of the stream. We measure V_K by the accuracy matrix

$$\mathbf{A} \in [0, 1]^{K \times K}, \quad A_{i,j} = \text{Acc}(V_j \text{ on } \mathcal{T}_i), \quad (11)$$

i.e. the accuracy on task i after training through task j . From $\mathbf{A}$ we report the three standard CL metrics of Lopez-Paz and Ranzato (2017):

$$\text{ACC} = \frac{1}{K} \sum_{i=1}^K A_{i,K}, \quad (12)$$

$$\text{BWT} = \frac{1}{K-1} \sum_{i=1}^{K-1} (A_{i,K} - A_{i,i}), \quad (13)$$

$$\text{FWT} = \frac{1}{K-1} \sum_{i=2}^K (A_{i,i-1} - \bar{a}_i), \quad (14)$$

where $\bar{a}_i$ is the accuracy of a random-init probe of the same architecture on $\mathcal{T}_i$ without any training.

6.3 Structural Guarantees of LAVA

The LAVA architecture — a frozen backbone M , a frozen feature extractor ϕ , and a bank of independent per-concept probes $\{P_k\}$ trained on disjoint $\mathcal{D}_k$ — yields three properties *by construction*, without any auxiliary regularizer (EWC, LwF, replay, etc.):

Proposition 6.1 (Zero backward interference). *Let V_k denote the LAVA verifier after training through task k . Since the parameters of probe P_i are fixed after task i and the backbone M, ϕ are frozen throughout, the prediction of P_i on any input is identical under V_i and V_K for all $K \geq i$. Hence*

$$A_{i,K} = A_{i,i} \quad \forall i \leq K, \quad \text{so BWT} = 0.$$

Proposition 6.2 (Constant per-task training cost). *Training task $\mathcal{T}_k$ requires only $\mathcal{O}(N_k)$ probe-parameter updates, independent of k . In contrast, a single shared probe with experience-replay or EWC requires either $\mathcal{O}(k)$ memory growth or an $\mathcal{O}(k)$ Fisher-information pass.*

Proposition 6.3 (Bounded inference overhead). *For a probe with d -dimensional input and a linear or shallow MLP head, the per-step verification*

cost of LAVA scales as $\mathcal{O}(Kd)$, which remains orders of magnitude smaller than one transformer forward pass for any practical K .

These three guarantees give LAVA its CL profile: zero forgetting, constant per-task supervision, and a flat inference cost curve.

6.4 Cumulative False-Accept Bound

Reviewer concern: as the number of active concepts K grows, does the probability of a step being incorrectly accepted by *all* probes blow up? Under conditional independence of probe errors given a truly invalid step, the joint false-accept probability satisfies

$$\Pr\left(\bigwedge_{k=1}^K P_k(f) \geq \tau_k \mid \text{step invalid for some } k\right) \leq \prod_{k=1}^K \text{FN}_k, \quad (15)$$

where FN_k is the per-probe false-negative rate at threshold τ_k . We empirically test the tightness of this bound and report it in Table 9.

6.5 Experimental Protocol

We organize the continual evaluation as four complementary experiments. Numbers in the tables below are placeholders (—) to be filled in.

Task stream. We use a stream of $K = 8$ verification concepts in the following order: MATH (correctness) $\rightarrow$ GSM8K (preference) $\rightarrow$ AIME (correctness) $\rightarrow$ WildJailbreak (safety) $\rightarrow$ StrongReject (safety) $\rightarrow$ TruthfulQA (truthfulness) $\rightarrow$ AlpacaEval (preference) $\rightarrow$ HumanEval (correctness). Splits follow Table 1.

Baselines. (i) **Joint multi-task probe** (upper bound, requires all $\mathcal{D}_k$ simultaneously). (ii) **Shared probe + naive fine-tuning** (CL lower bound). (iii) **Shared probe + EWC** (Kirkpatrick et al., 2017). (iv) **Shared probe + LwF** (Li and Hoiem, 2017). (v) **Replay-based shared probe** (small per-task buffer) (Lopez-Paz and Ranzato, 2017; Chaudhry et al., 2019). (vi) **LLM-as-judge** (SpecReason-style; no learning, $\mathcal{O}(K)$ inference cost). (vii) **Sequentially retrained PRM** (Qwen-Math-PRM, retrained on the union of past data at each step). (viii) **LAVA (ours)**.

6.5.1 Experiment A: Sequential Concept Learning

Train probes one task at a time on $\mathcal{T}_1, \dots, \mathcal{T}_8$. After each task, evaluate all prior tasks to populate $\mathbf{A}$.

Table 6: Sequential concept learning across $K = 8$ tasks. Higher ACC, FWT, and lower |BWT| (closer to 0) are better. Cost is per-task training time. **Placeholders to be filled.**

Method	ACC $\uparrow$	BWT	FWT $\uparrow$	Cost/task
Joint multi-task (upper bound)	—	0	—	$K \cdot 1k$
Shared probe + naive FT	—	—	—	1k
Shared probe + EWC (Kirkpatrick et al., 2017)	—	—	—	1k+Fisher
Shared probe + LwF (Li and Hoiem, 2017)	—	—	—	1k+KD
Replay (buffer 200/task) (Chaudhry et al., 2019)	—	—	—	1k+replay
LLM-as-judge (no learning)	—	0	0	0
Sequentially retrained PRM	—	—	—	$\sum k \cdot 500k$
LAVA (ours)	—	0 (Prop. 6.1)	—	1k (const)

Table 7: Within-concept distribution shift on mathematical correctness. We report probe accuracy on the current shard and on *previously seen* shards. **Placeholders to be filled.**

Regime	Probe at stage	GSM8K	MATH	AIME24	AIME25	HMMT25
Frozen	after GSM8K	—	—	—	—	—
Full retrain	at each stage	—	—	—	—	—
Incremental (LAVA)	per-shard probe	—	—	—	—	—

Report ACC/BWT/FWT and per-task wall-clock cost.

6.5.2 Experiment B: Backward / Forward Transfer Matrix

We visualize the full $\mathbf{A} \in \mathbb{R}^{8 \times 8}$ matrix as a heatmap (Figure to be added). For LAVA the matrix is exactly lower-triangular with $A_{i,K} = A_{i,i}$ on each row; for shared-probe baselines we expect off-diagonal decay (forgetting) and occasionally positive off-diagonals when concepts are related (transfer).

6.5.3 Experiment C: Within-Concept Distribution Shift

Fix the concept to *mathematical correctness* and present a stream of distributions of increasing difficulty: GSM8K $\rightarrow$ MATH $\rightarrow$ AIME 24 $\rightarrow$ AIME 25 $\rightarrow$ HMMT 25 (the last two contamination-controlled). We compare three regimes per shift: (a) *frozen* (no adaptation), (b) *full retrain* (train a fresh probe on the new shard only), and (c) *incremental* (replay-augmented or per-shard probe).

6.5.4 Experiment D: Base-Model Drift

Train the LAVA probe bank on backbone $M^{(1)}$ (e.g. Qwen2.5-1.5B-Instruct). Then switch to $M^{(2)}$ (e.g. Qwen2.5-1.5B-Instruct after a continued-RLHF round, or Qwen3-1.5B). We measure (i) zero-shot transfer of the existing probes to $M^{(2)}$ ’s activations and (ii) the sample budget needed for a 5-pt accuracy recovery via a small re-alignment step.

Table 8: Base-model drift. Accuracy after migration from $M^{(1)}$ to $M^{(2)}$. **Placeholders to be filled.**

Concept	Acc on $M^{(1)}$	Zero-shot on $M^{(2)}$	Samples to recover
MATH correctness	—	—	—
WildJailbreak safety	—	—	—
TruthfulQA	—	—	—

Table 9: Scaling with the number of simultaneously active concepts K . Empirical joint false-accept rate vs. the independence bound of Eq. 15. **Placeholders to be filled.**

K	Latency (ms/step)	Empirical FA	Bound $\prod_k \text{FN}_k$	Coverage	ECE
1	—	—	—	—	—
5	—	—	—	—	—
10	—	—	—	—	—
20	—	—	—	—	—
50	—	—	—	—	—

6.5.5 Experiment E: Scaling the Concept Count

We activate $K \in \{1, 5, 10, 20, 50\}$ probes simultaneously on a mixed evaluation stream. We report per-step inference latency, the empirical joint false-accept rate, and its independence-based upper bound from Eq. 15.

6.6 Related Continual Learning Literature

Our framing connects to three CL strands. **Regularization-based** methods such as EWC (Kirkpatrick et al., 2017), SI, and LwF (Li and Hoiem, 2017) prevent forgetting by penalizing drift of important parameters; LAVA instead enforces the constraint architecturally by freezing prior probes. **Replay-based** methods such as GEM (Lopez-Paz and Ranzato, 2017), A-GEM (Chaudhry et al., 2019), and iCaRL (Rebuffi et al., 2017) keep a buffer of past examples; LAVA avoids the storage cost entirely. **Modular / parameter-isolation** methods such as Progressive Networks (Rusu et al., 2016), AdapterFusion (Pfeiffer et al., 2021), and Progressive Prompts (Razdaibiedina et al., 2023) grow a separate module per task and are closest in spirit to LAVA’s probe bank — the novel contribution here is showing that this modular pattern is the *natural* architecture for step-level verification in speculative reasoning, not a CL afterthought.

6.7 Speculative Reasoning as a Downstream Application

The continually trained probe bank is directly usable as the parallel multi-concept verifier of Section 5.2. The efficiency results reported there

(~69% token reduction, 3–5× latency speedup) thus emerge as a downstream consequence of the CL formulation: once an organization has accumulated a probe bank over its deployment history, every speculative-reasoning rollout benefits from all accumulated probes *simultaneously and at constant inference cost*.

7 Conclusion

We recast step-level verification of LLM reasoning as a continual learning problem. The resulting framework, LAVA, represents the verifier as a bank of lightweight probes attached to a frozen backbone: each new concept is a new probe, prior probes are never touched, and inference activates all probes in parallel at constant per-step cost. This architectural choice gives zero backward interference, constant per-task training cost, and bounded inference overhead by construction. Deployed as the verifier in a speculative reasoning pipeline, the continually trained probe bank also recovers the efficiency gains (token reduction and latency speedup) over LLM/PRM-based verifiers as a downstream consequence rather than a separate design.

Limitations

Our continual-verification guarantees rest on the LLM backbone being frozen across the task stream. When the base model itself is updated (continued pre-training, additional RLHF rounds, or a checkpoint change), the activation geometry shifts and previously trained probes may become miscalibrated; we study this regime as Experiment D (Section 6.5.4), but a fully online re-alignment procedure is left to future work. Our zero-backward-interference guarantee (Prop. 6.1) also assumes each probe is trained independently — if probes are coupled (e.g. via shared low-rank adapters for forward transfer), backward interference may reappear and need explicit control. Finally, probe effectiveness still depends on alignment between probe-training data and the target domain; on knowledge-intensive tasks such as GPQA our probes underperform, suggesting that domain adaptation techniques and broader cross-architecture transferability studies are important directions.

Ethics Statement

This work has the potential to reduce the cost of multi-criteria verification and improve the practical

safety and reliability of reasoning systems, but it should be paired with careful evaluation, domain adaptation, and transparent reporting to avoid miscalibration and misuse. The paper may introduce potential risks. Because probe-based signals depend on training data and model internals, the verifier may fail under distribution shift, incorrectly accepting unsafe or incorrect steps or rejecting valid ones, especially in domains not represented in probe training. This could lead to over-trust in automated verification and downstream harms when systems are used in high-stakes settings (e.g., medical, legal, or security-related advice). There are also governance and transparency considerations. Hidden-state probing can be viewed as an internal monitoring mechanism; while it may improve safety, it can also create incentives to treat probe scores as definitive without sufficient interpretability or auditing. To mitigate these concerns, deployments should (i) report verification uncertainty, (ii) evaluate robustness across domains and adversarial inputs, (iii) calibrate thresholds to control false acceptance rates, and (iv) maintain fallbacks to stronger verification mechanisms when probe confidence is low.

References

- Guillaume Alain and Yoshua Bengio. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*.
- Yoshua Bengio and Yann LeCun. 2007. Scaling learning algorithms towards AI. In *Large Scale Kernel Machines*. MIT Press.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. 2019. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2023. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers

939	to solve math word problems. <i>arXiv preprint</i>	Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman	990
940	<i>arXiv:2110.14168</i> .	Banerjee, Huan Sun, Muhao Chen, and Chaowei	991
941	Ramzi Dakhmouche, Adrien Letellier, and Hossein	Xiao. 2025. AGrail: A lifelong agent guardrail	992
942	Gorji. 2025. Can linear probes measure llm uncer-	with effective and adaptive safety detection. <i>arXiv</i>	993
943	tainty? <i>arXiv preprint arXiv:2510.04108</i> .	<i>preprint arXiv:2502.11448</i> .	994
944	Yann Dubois, Balázs Galambosi, Percy Liang, and Tat-	Yun Luo, Zhicheng Yang, Fandong Meng, Youcheng	995
945	sunori B Hashimoto. 2024. Length-controlled al-	Li, Jie Zhou, and Yue Zhang. 2023. An empirical	996
946	pacaeval: A simple way to debias automatic evalua-	study of catastrophic forgetting in large language	997
947	tors. <i>arXiv preprint arXiv:2404.04475</i> .	models during continual fine-tuning. <i>arXiv preprint</i>	998
948	Neel Guha, Mayee Chen, Trevor Chow, Ishan Khare,	<i>arXiv:2308.08747</i> .	999
949	and Christopher Re. 2024. Smoothie: Label free	Bo Lv, Chen Tang, Yanan Zhang, Xin Liu, Yue Yu,	1000
950	language model routing. <i>Advances in Neural Infor-</i>	and Ping Luo. 2024. Specfuse: Ensembling large	1001
951	<i>mation Processing Systems</i> , 37:127645–127672.	language models via next-segment prediction. <i>arXiv</i>	1002
952	Tianjie Ju, Weiwei Sun, Wei Du, Xinwei Yuan,	<i>preprint arXiv:2412.07380</i> .	1003
953	Zhaochun Ren, and Gongshen Liu. 2024. How	Qichao Ma, Rui-Jie Zhu, Peiye Liu, Renye Yan, Fahong	1004
954	large language models encode context knowl-	Zhang, Ling Liang, Meng Li, Zhaoifei Yu, Zongwei	1005
955	edge? a layer-wise probing study. <i>arXiv preprint</i>	Wang, Yimao Cai, and 1 others. 2024. Inner-probe:	1006
956	<i>arXiv:2402.16061</i> .	Discovering copyright-related data generation in llm	1007
957	Muhammad Khalifa and 1 others. 2025. ThinkPRM:	architecture. <i>arXiv preprint arXiv:2410.04454</i> .	1008
958	A data-efficient long cot verifier for llm reasoning.	Alex Troy Mallen, Madeline Brumley, Julia	1009
959	<i>arXiv preprint</i> .	Kharchenko, and Nora Belrose. 2023. Elicit-	1010
960	James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz,	ing latent knowledge from "quirky" language models.	1011
961	Joel Veness, Guillaume Desjardins, Andrei A Rusu,	In <i>First Conference on Language Modeling</i> .	1012
962	Kieran Milan, John Quan, Tiago Ramalho, Ag-	Francesco Manigrasso, Stefan Schouten, Lia Morra,	1013
963	nieszka Grabska-Barwinska, and 1 others. 2017.	and Peter Bloem. 2024. Probing llms for logical	1014
964	Overcoming catastrophic forgetting in neural net-	reasoning. In <i>International Conference on Neural-</i>	1015
965	works. <i>Proceedings of the National Academy of</i>	<i>Symbolic Learning and Reasoning</i> , pages 257–278.	1016
966	<i>Sciences</i> , 114(13):3521–3526.	Springer.	1017
967	Zhizhong Li and Derek Hoiem. 2017. Learning without	Rohin Manvi, Anikait Singh, and Stefano Ermon. 2024.	1018
968	forgetting. <i>IEEE Transactions on Pattern Analysis</i>	Adaptive inference-time compute: LLMs can predict	1019
969	<i>and Machine Intelligence</i> , 40(12):2935–2947.	if they can do better, even mid-generation. <i>arXiv</i>	1020
970	Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri-	<i>preprint arXiv:2410.02725</i> .	1021
971	son Edwards, Bowen Baker, Teddy Lee, Jan Leike,	Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao	1022
972	John Schulman, Ilya Sutskever, and Karl Cobbe.	Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia.	1023
973	2023. Let’s verify step by step. In <i>The Twelfth Inter-</i>	2025. Towards efficient generative large language	1024
974	<i>national Conference on Learning Representations</i> .	model serving: A survey from algorithms to systems.	1025
975	Stephanie Lin, Jacob Hilton, and Owain Evans. 2021.	<i>ACM Computing Surveys</i> , 58(1):1–37.	1026
976	Truthfulqa: Measuring how models mimic human	Rui Pan, Yinwei Dai, Zhihao Zhang, Gabriele Oliaro,	1027
977	falsehoods. <i>arXiv preprint arXiv:2109.07958</i> .	Zhihao Jia, and Ravi Netravali. 2025. Specreason:	1028
978	Cong Liu, Xiaojun Quan, Yan Pan, Liang Lin, Weigang	Fast and accurate inference-time compute via specu-	1029
979	Wu, and Xu Chen. 2024. Cool-fusion: Fuse large	lative reasoning. <i>arXiv preprint arXiv:2504.07891</i> .	1030
980	language models without training. <i>arXiv preprint</i>	Henry Papadatos and Rachel Freedman. 2024. Lin-	1031
981	<i>arXiv:2407.19807</i> .	ear probe penalties reduce llm sycophancy. <i>arXiv</i>	1032
982	Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu	<i>preprint arXiv:2412.00967</i> .	1033
983	Li, Bqing Qi, Wanli Ouyang, and Bowen Zhou.	Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé,	1034
984	2025. Can 1b llm surpass 405b llm? rethinking	Kyunghyun Cho, and Iryna Gurevych. 2021.	1035
985	compute-optimal test-time scaling. <i>arXiv preprint</i>	AdapterFusion: Non-destructive task composition	1036
986	<i>arXiv:2502.06703</i> .	for transfer learning. In <i>Conference of the European</i>	1037
987	David Lopez-Paz and Marc’Aurelio Ranzato. 2017.	<i>Chapter of the Association for Computational Lin-</i>	1038
988	Gradient episodic memory for continual learning. In	<i>guistics</i> .	1039
989	<i>Advances in Neural Information Processing Systems</i> .	ACCURACY FROM QUESTION-ONLY LINEAR	1040
		PROBES. 2025. No answer needed: Predicting llm	1041
		answer accuracy from question-only linear probes.	1042
		<i>openreview</i> .	1043

1044	Alexandre Ramé, Guillaume Couairon, Mustafa Shukor,	Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan,	1097
1045	Corentin Dancette, Jean-Baptiste Gaya, Laure	Thuy-Trang Vu, and Gholamreza Haffari. 2024. Con-	1098
1046	Soulier, and Matthieu Cord. 2024. WARM: On the	tinual learning for large language models: A survey.	1099
1047	benefits of weight averaged reward models. <i>arXiv</i>	<i>arXiv preprint arXiv:2402.01364</i> .	1100
1048	<i>preprint arXiv:2401.12187</i> .		
1049	Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Ma-	Seohong Xu, Adam Foster, Marc’ Aurelio Ranzato, and	1101
1050	dian Khabisa, Mike Lewis, and Amjad Almahairi.	Oriol Vinyals. 2024. Right now, wrong then: Non-	1102
1051	2023. Progressive prompts: Continual learning for	stationary direct preference optimization under pref-	1103
1052	language models. In <i>International Conference on</i>	erence drift. <i>arXiv preprint arXiv:2407.18676</i> .	1104
1053	<i>Learning Representations</i> .		
1054	Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg	Yangyifan Xu, Jianghao Chen, Junhong Wu, and Jiajun	1105
1055	Sperl, and Christoph H Lampert. 2017. iCaRL: In-	Zhang. 2025. Hit the sweet spot! span-level ensem-	1106
1056	cremental classifier and representation learning. In	ble for large language models. In <i>Proceedings of</i>	1107
1057	<i>IEEE Conference on Computer Vision and Pattern</i>	<i>the 31st International Conference on Computational</i>	1108
1058	<i>Recognition</i> .	<i>Linguistics</i> , pages 8314–8325.	1109
1059	Andrei A Rusu, Neil C Rabinowitz, Guillaume Des-	Biao Yi, Tiansheng Huang, Sishuo Chen, Tong Li, Zheli	1110
1060	jardins, Hubert Soyer, James Kirkpatrick, Koray	Liu, Zhixuan Chu, and Yiming Li. 2025. Probe be-	1111
1061	Kavukcuoglu, Razvan Pascanu, and Raia Hadsell.	fore you talk: Towards black-box defense against	1112
1062	2016. Progressive neural networks. <i>arXiv preprint</i>	backdoor unalignment for large language models.	1113
1063	<i>arXiv:1606.04671</i> .	<i>arXiv preprint arXiv:2506.16447</i> .	1114
1064	Swarnadeep Saha, Archiki Prasad, Justin Chen, Peter	Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Au-	1115
1065	Hase, Elias Stengel-Eskin, and Mohit Bansal. 2025.	rojit Panda, Jinyang Li, and He He. 2025. Rea-	1116
1066	System 1. x: Learning to balance fast and slow plan-	soning models know when they’re right: Probing	1117
1067	ning with language models. In <i>The Thirteenth Inter-</i>	hidden states for self-verification. <i>arXiv preprint</i>	1118
1068	<i>national Conference on Learning Representations</i> .	<i>arXiv:2504.05419</i> .	1119
1069	et al. Schubert. 2026. Just-in-time reinforcement learn-	Di Zhang, Xiaoshui Huang, Dongzhan Zhou, Yuqiang	1120
1070	ing: Continual learning in LLM agents without gra-	Li, and Wanli Ouyang. 2024. Accessing gpt-4 level	1121
1071	dient updates. <i>arXiv preprint arXiv:2601.18510</i> .	mathematical olympiad solutions via monte carlo	1122
1072	Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen,	tree self-refine with llama-3 8b. <i>arXiv preprint</i>	1123
1073	and Yang Zhang. 2024. "do anything now": Charac-	<i>arXiv:2406.07394</i> .	1124
1074	terizing and evaluating in-the-wild jailbreak prompts	Andy Zou, Long Phan, Sarah Chen, James Campbell,	1125
1075	on large language models. In <i>Proceedings of the</i>	Phillip Guo, Richard Ren, Alexander Pan, Xuwang	1126
1076	<i>2024 on ACM SIGSAC Conference on Computer and</i>	Yin, Mantas Mazeika, Ann-Kathrin Dombrowski,	1127
1077	<i>Communications Security</i> , pages 1671–1685.	and 1 others. 2023. Representation engineering: A	1128
1078	Alexandra Souly, Qingyuan Lu, Dillon Bowen,	top-down approach to ai transparency. <i>arXiv preprint</i>	1129
1079	Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel,	<i>arXiv:2310.01405</i> .	1130
1080	Justin Svegliato, Scott Emmons, Olivia Watkins, and	A Notation Summary	1131
1081	1 others. 2024. A strongreject for empty jailbreaks.	• $\mathcal{V}$: vocabulary; $\mathcal{V}^*$: token sequences.	1132
1082	<i>Advances in Neural Information Processing Systems</i> ,	• x : input query; r_i : i -th reasoning step; n :	1133
1083	37:125416–125440.	number of steps.	1134
1084	et al. Wang. 2025. Online process reward learning	• $C_i := [x, r_1, \dots, r_i]$: context prefix before	1135
1085	for agentic reinforcement learning. <i>arXiv preprint</i>	proposing step $i+1$.	1136
1086	<i>arXiv:2509.19199</i> .	• M_w, M_s : draft and strong autoregressive	1137
1087	Xiao Wang, Yuansen Zhang, Tianze Chen, Songyang	LMs.	1138
1088	Gao, Senjie Jin, Xianjun Yang, Zhiheng Xi, Rui	• $V(\cdot)$: verification mechanism; K : number of	1139
1089	Zheng, Yicheng Zou, Tao Gui, Qi Zhang, and Xuan-	verification concepts.	1140
1090	jing Huang. 2023. TRACE: A comprehensive bench-	• τ_k : acceptance threshold for concept k .	1141
1091	mark for continual learning in large language models.	B Algorithm	1142
1092	In <i>arXiv preprint arXiv:2310.06762</i> .	To better illustrate the pipeline, we also add an	1143
1093	Yongjie Wang, Yibo Wang, Xin Zhou, and Zhiqi Shen.	algorithm of using LAVA for inference here.	1144
1094	2025. Response uncertainty and probe modeling:		
1095	Two sides of the same coin in llm interpretability?		
1096	<i>arXiv preprint arXiv:2505.18575</i> .		

Algorithm 1 LAVA Inference

Require: Query x , draft model M_w , strong model M_s , probes $\{P_k\}_{k=1}^K$, thresholds $\{\tau_k\}_{k=1}^K$

Ensure: Complete reasoning chain $\mathcal{R}$

```
1: Initialize context  $C_0 := [x]$ 
2: Initialize reasoning chain  $\mathcal{R} := []$ 
3:  $i \leftarrow 0$ 
4: while reasoning not complete do
5:   // Step 1: Draft step generation
6:    $r_{i+1}^w \sim M_w(\cdot \mid C_i)$   $\triangleright$  Eq. (1)
7:   // Step 2: Feature extraction
8:   Extract hidden states  $\{h_t^{(l)}\}_{t=1}^T$  from  $M_w$ 
   for  $r_{i+1}^w$ 
9:     Compute feature vector:  $f :=$ 
        $[h_{T-4}^{(l*)}, \dots, h_T^{(l*)}]$   $\triangleright$  Eq. (3)
10:  // Step 3: Multi-concept verification
11:  for  $k = 1$  to  $K$  do
12:     $\text{score}_k \leftarrow P_k(f) = \sigma(w_k^\top f + b_k)$   $\triangleright$ 
      Eq. (4)
13:  end for
14:  // Step 4: Gating decision
15:   $\text{accept} \leftarrow \bigwedge_{k=1}^K (\text{score}_k \geq \tau_k)$   $\triangleright$  Eq. (6)
16:  if  $\text{accept}$  then
17:     $r_{i+1} \leftarrow r_{i+1}^w$   $\triangleright$  Accept draft
18:  else
19:     $r_{i+1} \sim M_s(\cdot \mid C_i)$   $\triangleright$  Strong
      regeneration
20:  end if
21:  // Step 5: Update context
22:   $\mathcal{R}.\text{append}(r_{i+1})$ 
23:   $C_{i+1} \leftarrow C_i + [r_{i+1}]$ 
24:   $i \leftarrow i + 1$ 
25: end while
26: return  $\mathcal{R}$ 
```

C Evaluation for AlpacaEval

AlpacaEval (Dubois et al., 2024) compares generations from two models (a candidate model and a reference model) and lets another LLM-as-a-judge judge which generation is better, then computes a length-controlled win rate. For a fair comparison, we adopt a fixed reference model (Davinci 003, the default model for AlpacaEval). For the judge, we adopt Claude v2. According to (Dubois et al., 2024), Claude v2 shows the state-of-the-art performance (65.3) aligning with human preference.

D Implementation Details

We use the following prompt template for evaluation:

“Evaluate the last reasoning step solely based on factual correctness and logical validity. Ignore style, phrasing, or overall usefulness—only judge whether the step is objectively correct and logically follows from prior steps. Assign a score from 0 to 9.”

Due to the 4096 token context length limitation of Skywork-PRM, we implemented truncation when generations exceeded this threshold. This constraint significantly impacted overall performance, as numerous solution attempts were prematurely terminated before completion, preventing the model from generating full solutions to the given problems.

E More Related Works

Interpretability Probes in Neural Networks. Probes (Alain and Bengio, 2016) have emerged as a fundamental interpretability technique for understanding the internal representations of deep neural networks. Probes typically extract features from intermediate layers and map them to classification outputs, thereby providing insights into the quality and nature of learned representations at different depths within the network. The effectiveness of probe-based extraction serves as a metric for evaluating how well specific information is encoded in neural representations, offering valuable perspectives on the specialization of different layers and the types of features captured during training. Eliciting Latent Knowledge (ELK) (Mallen et al., 2023) has shown that linear probes can reliably extract truthful information from model activations, even

Table 10: Performances on Mixture of Benchmark. SR is short for SpecReason.

Math& Safety	Correctness (pass@1)	Safety Ratio (%)	False Unsafe (%)	False Incorrect (%)	Latency (s)	Tokens (n)
SR	82	56	11	14	192	1232
LAVA (probe parallel)	82	59	19	12	100	762
LAVA (probe refined)	84	59	9	12	102	791

Math& Truthfulness	Correctness (pass@1)	Truthfulness (%)	False Fake (%)	False Incorrect (%)	Latency (s)	Tokens (n)
SR	82	56	20	12	82	1062
LAVA (probe parallel)	82	64	32	18	62	721
LAVA (probe refined)	83	62	10	12	60	708

when the LLM has been trained to produce deceptive outputs in certain contexts. This finding suggests that internal representations may preserve accurate knowledge despite surface-level manipulations in the training process. Contemporary approaches have extended probe-based analysis to generation evaluation tasks. For instance, recent work (Zhang et al., 2025) has proposed parsing model generations into discrete chunks and employing probes to assess whether each chunk contains a correct final answer. Similarly, LLM safety research (Zou et al., 2023) has demonstrated that individual probes can effectively detect the presence of sleeper agents, highlighting their utility in identifying potentially harmful model behaviors. The interpretability afforded by probes extends to various linguistic phenomena, including syntactic structures (such as part-of-speech tagging), semantic relationships, and world knowledge representations. This broad applicability has established probe-based analysis as an essential tool for understanding the internal mechanisms of complex models, particularly in natural language processing and computer vision domains where learned representations often remain opaque to direct interpretation.

Inference-Time Scaling. Current inference-time scaling (ITS) approaches typically employ three primary strategies: parallel scaling, sequential scaling, and hybrid scaling methods. These techniques stimulate LLMs to generate multiple or extended samples, subsequently employing verification mechanisms to either select optimal reasoning paths or guide the reasoning process through established criteria. Existing verification approaches predominantly rely on two categories of methods: trained process reward models (PRMs) (Liu et al.,

2025) and LLM-as-a-judge frameworks (Chen et al., 2021). The latter may utilize either the generating model itself or more capable, larger-scale language models for evaluation purposes. However, both PRM and LLM-as-a-judge approaches operate exclusively on discrete token representations, potentially overlooking valuable information encoded in continuous hidden representations. A critical limitation of current ITS methods lies in their exclusive reliance on discrete tokens, thereby neglecting the rich information contained within high-dimensional continuous hidden representations. The standard LLM generation process involves projecting hidden representations to probability distributions—a step that may result in semantic information loss—followed by sampling procedures that discard substantial amounts of encoded information. This observation motivates alternative approaches that leverage continuous representations for improved inference-time reasoning. Adaptive inference time compute (Manvi et al., 2024) also finds that the model itself can predict if itself can do better with restart.

F A Mini-Survey of Continual Learning for LLM Verification

This section surveys the small but rapidly growing set of works that touch the intersection of (i) continual / lifelong learning and (ii) verification, reward modeling, or safety filtering of LLM outputs. Our goal is not to catalogue all of continual learning — recent surveys do this well (Wu et al., 2024) — but to position LAVA against the specific subset of methods that target a moving *verifier* target. Table 11 summarizes the works we discuss; each row is referenced in the text below.

F.1 Continual-Learning Benchmarks for LLMs

The current standard benchmark for CL in LLMs is **TRACE** (Wang et al., 2023), which composes 8 tasks across domain QA (C-STANCE, FOMC, MeetingBank), multilingual, code (Py150), and math (GSM8K). TRACE established the now-canonical finding that supervised fine-tuning on a new task can erase tens of percentage points of accuracy on previously aligned skills (the GSM8K drop from 43.14% to 2.12% on Llama-2-Chat-13B is the most-cited number). The companion survey of Wu et al. (2024) extends this picture by organizing CL methods for LLMs into pretraining, instruction-tuning, and alignment stages, and Luo et al. (2023) measure how forgetting scales with model size in the 1B–7B range.

These benchmarks all target the *whole LLM*: the unit of continual learning is the (very large) parameter set being fine-tuned, and forgetting is measured as accuracy degradation on general ability. None of them measure forgetting of a *verifier* attached to a frozen LLM, which is the regime LAVA inhabits.

F.2 Continual / Drift-Aware Reward and Process-Reward Models

The reward-model literature has begun to engage with CL implicitly, under the label of “reward drift” or “policy drift.” **WARM** (Ramé et al., 2024) averages multiple independently-fine-tuned RMs in weight space to absorb the distribution shift induced by policy updates during RLHF. **Non-stationary DPO** (Xu et al., 2024) explicitly assumes a time-varying preference distribution and adapts the DPO objective accordingly. **Online Process Reward Learning** (Wang, 2025) re-trains the PRM jointly with the policy in agentic RL, sidestepping the offline labeling cost. **ThinkPRM** (Khalifa

et al., 2025) pushes PRM data-efficiency in the offline direction with a verbalized-CoT verifier.

A common thread is that all of these methods still update a *single shared* verifier — either via averaging (WARM), online retraining (Online PRM), or replay (Non-stationary DPO). None provide a structural zero-forgetting guarantee; their forget metrics are downstream (win-rate, RM reliability) rather than direct per-task accuracy retention.

F.3 Lifelong Safety Filtering

AGrail (Luo et al., 2025) is the closest precedent in spirit to LAVA: a “lifelong agent guardrail” that adaptively generates new safety checks and maintains a memory module of past attacks. AGrail evaluates on a new **Safe-OS** benchmark spanning systemic, environmental, and task-specific risks, and demonstrates 0% Attack Success Rate against prompt injection with 95.6% benign-action preservation. AGrail differs from LAVA in two important ways: (i) its “checks” are generated and verified by LLM calls — inference cost grows with the size of the check library — whereas LAVA’s probes have $\mathcal{O}(d)$ per-step cost; and (ii) AGrail does not formalize a continual-learning evaluation protocol (no BWT/FWT) — its lifelong claim is operational rather than measured.

F.4 Gap Analysis and LAVA’s Position

Cross-referencing Table 11 reveals a consistent gap:

- Whole-LLM CL benchmarks (TRACE, the survey, empirical-forgetting study) target the wrong unit (the LLM, not the verifier).
- Drift-aware RM/PRM work (WARM, Non-stationary DPO, Online PRM) treats the verifier as a single shared model, accepting forgetting as a cost to be mitigated rather than eliminated.
- Lifelong safety work (AGrail) demonstrates lifelong operation but at LLM-call inference cost and without standard CL metrics.
- No prior work, to our knowledge, provides a verifier that simultaneously satisfies: (i) per-task supervision $\mathcal{O}(10^3)$, (ii) zero forgetting by construction, (iii) inference cost flat in the number of monitored concepts, and (iv) standard CL evaluation (ACC / BWT / FWT) on a stream of verification tasks.

LAVA’s probe-bank architecture fills exactly this gap, and our continual-learning protocol of Section 6.5 (sequential concept arrival, transfer matrix, within-concept shift, base-model drift, K -scaling)

is designed to be reusable as the first CL benchmark dedicated to step-level LLM verification.

F.5 Benchmarks We Adopt or Extend

Based on the above, our evaluation suite (Section 6.5) borrows from each strand:

- **From TRACE** (Wang et al., 2023): the sequential-task evaluation protocol and the ACC/BWT/FWT metrics, applied to verification rather than to LLM ability.
- **From AGrail / Safe-OS** (Luo et al., 2025): the lifelong safety scenario — adapted to the probe setting by introducing a new safety probe per attack family rather than re-training a single guardrail.
- **From WARM / Non-stationary DPO** (Ramé et al., 2024; Xu et al., 2024): the “base policy / base model drift” axis, which we instantiate as Experiment D (Section 6.5.4).
- **From the standard CL literature** (Kirkpatrick et al., 2017; Li and Hoiem, 2017; Lopez-Paz and Ranzato, 2017; Chaudhry et al., 2019; Rebuffi et al., 2017; Rusu et al., 2016): the baselines (EWC, LwF, GEM, A-GEM, iCaRL, Progressive Networks) against which we compare.

Table 11: Survey of continual-learning work most relevant to LLM verification. “Target of CL” is the component that is updated over time; “Eval” lists the headline benchmark(s); “Forget metric” is the primary forgetting/transfer measurement used.

Work	Year	Motivation	Target of CL	Eval / Bench- mark	Forget metric
TRACE (Wang et al., 2023)	2023	Aligned LLMs (7B–70B) lose massive general ability after fine-tuning on new tasks (GSM8K acc. drops 43% → 2%).	Whole LLM (instruction-tuned)	8 tasks: domain QA, multilingual, code, math (incl. C-STANCE, FOMC, Meeting-Bank, GSM8K).	Overall Performance, Backward Transfer, General Ability Delta.
Luo et al. (Luo et al., 2023)	2023	Empirically characterize catastrophic forgetting in continually fine-tuned decoder LLMs (1B–7B).	Whole LLM (decoder)	Domain knowledge + reasoning + reading comprehension suites.	Per-task accuracy drop vs. scale.
CL for LLMs Survey (Wu et al., 2024)	2024	First survey of CL specifically for LLMs; introduces a multi-stage taxonomy (pretraining → instruction-tuning → alignment).	Whole LLM at every stage	Coverage paper, no headline benchmark.	N/A (taxonomy).
WARM (Ramé et al., 2024)	2024	Reward models drift under policy updates and disagree under noisy labels, enabling reward hacking.	Reward model (RM)	Summarization, RLHF dialog.	RM reliability under distribution shift; downstream win-rate.
Non-stationary DPO (Xu et al., 2024)	2024	Preferences are not stationary — they shift with new information, demographics, and culture.	Preference / DPO policy	Synthetic + Anthropic HH with simulated drift.	Win-rate over drifting preference horizon.
AGrail (Luo et al., 2025)	2025	Single-pass safety classifiers (e.g. LlamaGuard) fail under adaptive attacks and cannot absorb new policies.	Safety check generator + memory module	Safe-OS (3 attack types) + AdvWeb + EIA.	ASR after lifelong updates; benign-action preservation.
Online PRM Learning (Wang, 2025)	2025	Offline PRMs become stale as the policy improves; require expensive re-labeling.	PRM + policy (joint)	Agentic RL benchmarks (WebShop, ALF-World).	Stability of step rewards vs. policy iteration.
ThinkPRM (Khalifa et al., 2025)	2025	Discriminative PRMs need $\mathcal{O}(10^5)$ step labels; want a data-efficient verifier.	Verbalized CoT verifier (generative PRM)	ProcessBench, MATH-shepherd.	N/A (no explicit CL); but motivates low-data verifier updates.
JIT-RL (Schubert, 2026)	2026	Continual learning in deployed LLM agents without gradient updates — use in-context memory instead.	Agent’s working memory (no weight updates)	Long-horizon agent tasks (custom).	Cumulative reward over deployment horizon.
LAVA (ours)	2026	New concepts arrive continuously; PRM retraining forgets, LLM-judge inference cost grows linearly.	Probe bank (frozen backbone, frozen old probes, one new probe per task)	8-task stream of correctness / safety / truthfulness / preference / code concepts (Section 6.2).	ACC, BWT ($\equiv 0$ by construction), FWT; cumulative FA bound.